파이썬 완전 백과사전 2권 · 딥러닝 기초 교안

2일차

모든 모델이 따르는 표준형, 데이터가 흘러가는 길

6~10차시 · 책 6장 ~ 7장 7.6절

한 차시 50분 · 하루 5차시 · 예제: examples/vol2/06-nn-module, 07-data-pipeline

오늘의 시간표

차시	주제	책 범위
6	층, 활성화 함수, nn.Module, 파라미터	6장 들어가며 ~ 6.4
7	손실 함수, 옵티마이저, 학습 루프 다섯 줄	6.5 ~ 6.7
8	평가 루프, 장치, 저장, FashionMNIST 실습	6.8 ~ 6.11
9	Dataset, TensorDataset, DataLoader	7장 들어가며 ~ 7.3
10	CSV·이미지 폴더 Dataset, transforms	7.4 ~ 7.6

한 차시 50분: 도입 5분 · 설명과 시연 30분 · 직접 실행 10분 · 정리 5분

준비: examples/vol2에서 uv sync · 6장 예제는 대부분 1~2초, FashionMNIST 세 파일만 13~20초

6차시

06

층과 활성화 함수, nn.Module

책 6장 들어가며 ~ 6.4절

학습 목표

- 층 하나가 **입력 @ 가중치 + 편향**임을 손으로 계산해 nn.Linear와 맞춘다
- 활성화 함수가 왜 필요한지 XOR 문제로 확인한다
- nn.Module로 모델을 만들고 파라미터를 센다

6장 모델이 달라도 학습하는 모양은 같다

요리 학원은 메뉴가 매일 바뀌어도 수업 순서는 같다: 손질하고, 조리하고, **맛보고, 고치고**, 다시 맛본다

모델 · nn.Module

입력을 받아 예측을 낸다. 층을 쌓아서 만든다

손실 함수 · nn.MSELoss, nn.CrossEntropyLoss

예측이 얼마나 틀렸는지 숫자 하나로 잰다

옵티마이저 · torch.optim.SGD, Adam

기울기를 보고 파라미터를 고친다

학습 루프

위의 셋을 정해진 순서로 돌리는 **다섯 줄**짜리 반복문

6.1 층 하나: 입력 @ 가중치 + 편향

```
x = torch.tensor([[0.2, 0.5, 0.1], [0.9, 0.1, 0.4], [0.4, 0.8, 0.7], [0.6, 0.3, 0.2]])  # 손님 4명 × 특성 3개
torch.manual_seed(0)
가중치 = torch.randn(3, 2)           # 입력 3개 → 출력 2개
편향 = torch.zeros(2)                # 출력마다 하나씩
출력 = x @ 가중치 + 편향              # (4, 3) @ (3, 2) + (2,) → (4, 2)
print(출력)
print("모양:", 출력.shape)
print((x[0] * 가중치[:, 0]).sum() + 편향[0])     # 첫 손님의 첫 번째 출력을 풀어 쓰면
```

```
tensor([[-0.8896,  0.0857],
        [ 0.7352, -0.7667],
        [-1.8858, -0.6416],
        [ 0.0541, -0.2852]])
모양: torch.Size([4, 2])
tensor(-0.8896)
```

가중치: 입력을 얼마나 중요하게 볼지 · 3 × 2 = 6개
편향: 입력과 상관없이 더하는 기본값 · 출력마다 1개

이런 계산 한 덩어리가 **층**, 학습으로 고칠 숫자가 **파라미터**

6.1 nn.Linear는 같은 일을 하는 부품

```
층 = nn.Linear(3, 2)                  # 입력 3개 → 출력 2개
print("weight:", 층.weight.shape, "bias:", 층.bias.shape)
x = torch.rand(4, 3)
직접 = x @ 층.weight.T + 층.bias     # 손으로 같은 계산
print("같은가:", torch.allclose(층(x), 직접))
print("학습 대상인가:", 층.weight.requires_grad)
```

```
weight: torch.Size([2, 3]) bias: torch.Size([2])
같은가: True
학습 대상인가: True
```

weight 모양은 **(출력, 입력)** = (2, 3) → 손으로 하려면 **.T**

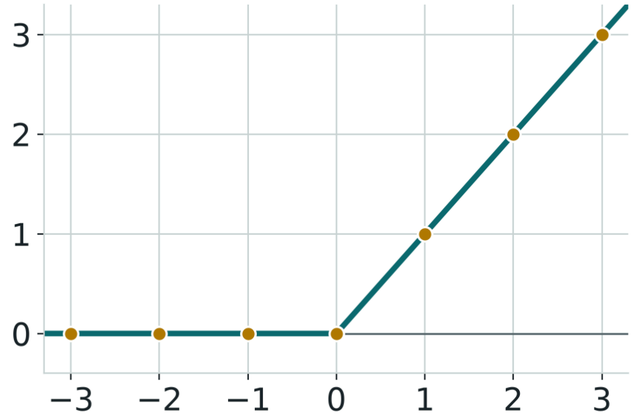
선형층 백 개를 쌓아도 선형층 **하나**와 같다: 2 × (3 × x) = 6 × x

```
두_층 = 층2(층1(x))                    # 3 → 5 → 2
W = 층2.weight @ 층1.weight            # 두 층을 합친 가중치 하나
b = 층2.weight @ 층1.bias + 층2.bias
한_층 = x @ W.T + b
print("같은가:", torch.allclose(두_층, 한_층, atol=1e-6))   # 같은가: True
```

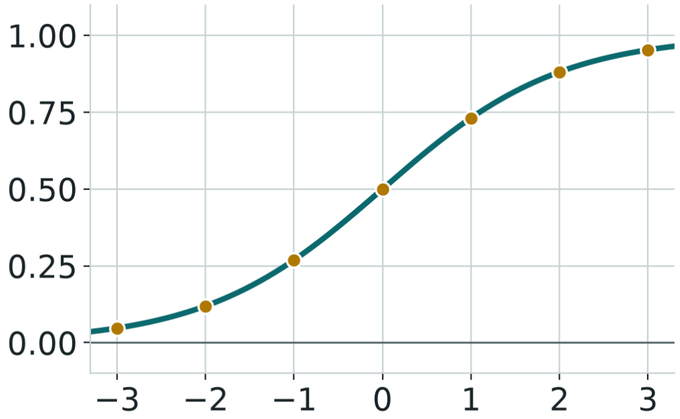
6차시 · 6.1 nn.Linear, 층을 그냥 쌓으면 · 예제 06-nn-module/b02_linear.py, b03_stack_linear.py (핵심 부분)

6.2 활성화 함수: 직선을 구부리는 장치

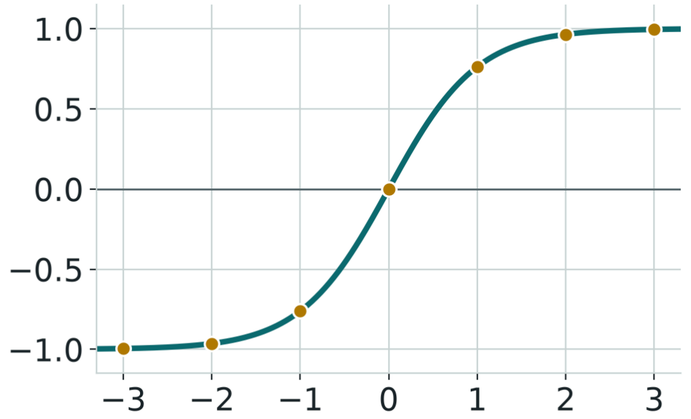
ReLU: 음수는 0, 양수는 그대로



시그모이드: 0과 1 사이로 눌러 넣기



tanh: 같은 모양, 범위는 -1 ~ 1



은닉층 사이에는 일단 ReLU: 규칙이 단순해 빠르고, 꺾인 조각을 겹치면 어떤 곡선이든 흉내 낸다

시그모이드·소프트맥스는 **맨 끝에서 확률을 읽을 때**, 활성화 함수에는 파라미터가 없다

6.2 소프트맥스: 점수를 확률로

```
# 사진 2장 × 클래스 3개(고양이, 개, 새)의 점수
로짓 = torch.tensor([[2.0, 1.0, 0.1],
                     [0.5, 0.5, 3.0]])
확률 = torch.softmax(로짓, dim=1)    # 클래스 방향으로 합이 1
print(확률)
print("줄마다 합:", 확률.sum(dim=1))
print("고른 클래스:", 확률.argmax(dim=1), 로짓.argmax(dim=1))
print(torch.softmax(로짓 * 10, dim=1).round(decimals=4))
print(torch.softmax(torch.tensor([1000.0, 0.0]), dim=0))
```

```
tensor([[0.6590, 0.2424, 0.0986],
        [0.0705, 0.0705, 0.8590]])
줄마다 합: tensor([1.0000, 1.0000])
고른 클래스: tensor([0, 2]) tensor([0, 2])
tensor([[1., 0., 0.],
        [0., 0., 1.]])
tensor([1., 0.])
```

확률로 바꿔도 **순서는 그대로**: 고르기만 할 땐 로짓에 바로 argmax

차이를 10배로 키우면 0과 1로 쏠린다: 부드러운 **max**

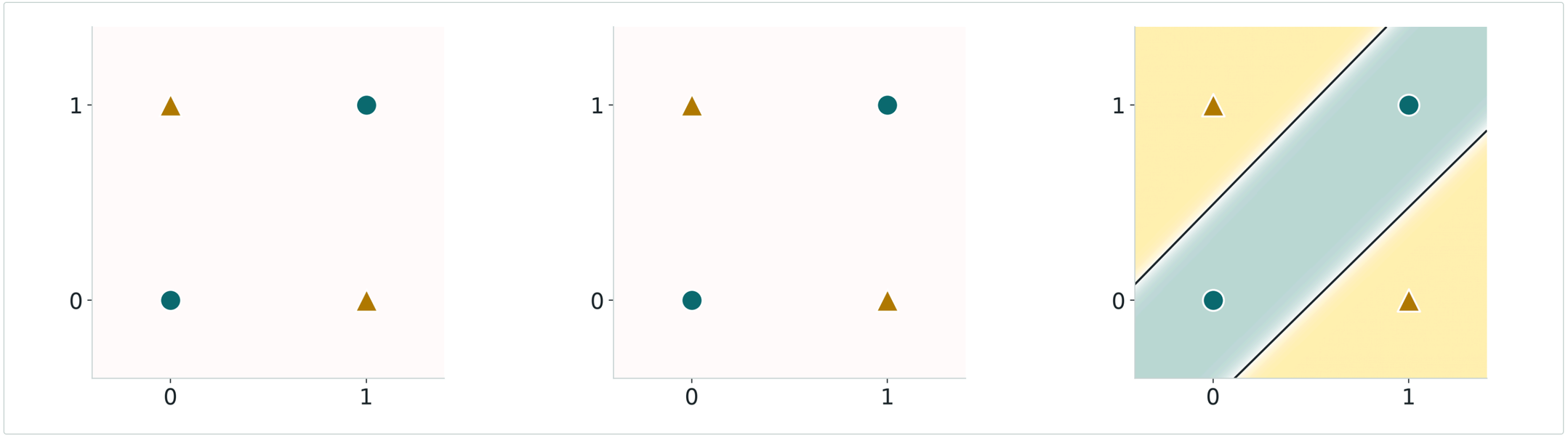
[1000, 0]도 안전: 직접 짜지 말고 **torch.softmax**

6.2 직선으로는 못 긋는 경계: XOR

선형 한 층
확률 [0.5, 0.5, 0.5, 0.5] · 맞힘 2/4

선형 두 층
확률 [0.5, 0.5, 0.5, 0.5] · 맞힘 2/4

선형 + ReLU + 선형
확률 [0.0, 1.0, 1.0, 0.0] · 맞힘 **4/4**



청록 ● 정답 0 · 노랑 ▲ 정답 1 · 가운데 **ReLU 하나**가 공간을 접어 직선으로 못 긋던 경계를 긋는다

6차시 · 6.2 XOR · 예제 06-nn-module/b06_xor.py와 같은 설정(Adam lr=0.05, 2000단계, 씨앗 1)으로 평면을 색칠한 그림

6.3 nn.Module로 설계도 그리기

```
class 작은신경망(nn.Module):
    def __init__(self, 입력수: int, 은닉수: int, 출력수: int) -> None:
        super().__init__()              # 장부부터 마련
        self.층1 = nn.Linear(입력수, 은닉수)
        self.활성화 = nn.ReLU()
        self.층2 = nn.Linear(은닉수, 출력수)
    def forward(self, x: torch.Tensor) -> torch.Tensor:
        x = self.층1(x)
        x = self.활성화(x)
        return self.층2(x)
```

```
모델 = 작은신경망(입력수=4, 은닉수=8, 출력수=3)
print(모델)
x = torch.rand(5, 4)
출력 = 모델(x)    # forward를 직접 부르지 않는다
print(출력.shape)
```

__init__: 쓸 부품 준비
forward: 입력이 지나가는 길
부를 때는 **모델(x)**: __call__이 준비 후 forward

```
작은신경망(
  (층1): Linear(in_features=4, out_features=8, bias=True)
  (활성화): ReLU()
  (층2): Linear(in_features=8, out_features=3, bias=True)
)
torch.Size([5, 3])
```

6.3 nn.Sequential, 그리고 리스트에 담으면 사라지는 층

```
사진_모델 = nn.Sequential(
    nn.Flatten(),          # (묶음, 1, 28, 28) → (묶음, 784)
    nn.Linear(28 * 28, 64),
    nn.ReLU(),
    nn.Linear(64, 10),
)
사진들 = torch.rand(32, 1, 28, 28)
print(사진_모델[0](사진들).shape, "→", 사진_모델(사진들).shape)
```

```
torch.Size([32, 784]) → torch.Size([32, 10])
```

줄 세우기만 하면 **Sequential**, 갈림길·되풀이·if가 있으면 **클래스**. 실무는 둘을 섞는다

```
self.층들 = [nn.Linear(4, 4) for _ in range(3)]                # 그냥 리스트
self.층들 = nn.ModuleList([nn.Linear(4, 4) for _ in range(3)])  # 등록되는 리스트
```

```
리스트모델: 파라미터 0개, 출력 (2, 4)
모듈리스트모델: 파라미터 60개, 출력 (2, 4)
```

오류 없이 돌아가는데 **영원히 학습되지 않는다**: 옵티마이저·.to()·저장에서 빠진다

6.4 파라미터 들여다보기

```
모델 = nn.Sequential(nn.Flatten(), nn.Linear(784, 256),
                     nn.ReLU(), nn.Linear(256, 10))
for 이름, p in 모델.named_parameters():
    print(f"{이름:9} {str(tuple(p.shape)):11} {p.numel():>8,}개")
전체 = sum(p.numel() for p in 모델.parameters())

for p in 모델[1].parameters():      # 첫 층 얼리기
    p.requires_grad_(False)
```

```
1.weight  (256, 784)   200,704개
1.bias    (256,)           256개
3.weight  (10, 256)      2,560개
3.bias    (10,)             10개
전체 203,530개 = 203,530개
float32로 약 0.78MB
학습 대상 2,570개
```

nn.Linear(입력, 출력)은
입력 × 출력 + 출력개

첫 층 하나가 **98%** 넘게: 사진을 펴서 784개를 모두 연결 (9장)

얼린 파라미터는 학습해도 그대로: **전체**와 **학습 대상**을 나눠 센다 (10장)

6차시 · 6.4 파라미터 들여다보기 · 예제 06-nn-module/b09_params.py (핵심 부분)

직접 실행 10분

1. b06_xor.py를 실행한 뒤, 세 번째 모델의 nn.ReLU()를 **nn.Sigmoid()**로 바꿔 다시 실행한다. 네 점을 다 맞힐까?
2. b09_params.py의 은닉층 256을 **512**로 바꾸기 전에 전체 파라미터 수를 손으로 계산하고, 실행해 맞춰 본다
3. b10_modulelist.py의 range(3)을 range(5)로 바꾸면 두 모델의 파라미터는 각각 몇 개일까?

```
cd examples/vol2/06-nn-module && uv run python b06_xor.py
```

nn.Linear(784, 128) 층 하나의 파라미터는 몇 개일까요?

Q

가 100,352개

나 100,480개

다 912개

라 784개

6차시 정리

층 = 입력 @ 가중치 + 편향. nn.Linear의 weight는 (출력, 입력), 선형층만 쌓으면 하나와 같다

층 사이의 **활성화 함수**(은닉층은 ReLU)가 직선으로 못 긋는 경계를 긋게 한다

nn.Module: __init__에 부품, forward에 길, 부를 때는 모델(x). 층 목록은 ModuleList

다음 차시: 얼마나 틀렸나(손실), 어떻게 고칠까(옵티마이저), 다섯 줄 학습 루프

7차시

07

손실 함수, 옵티마이저, 학습 루프

책 6.5 ~ 6.7절

학습 목표

- 문제에 맞는 손실 함수를 고르고, **로짓**을 넣어야 하는 이유를 말한다
- step()이 하는 일과 SGD·모멘텀·Adam의 차이를 결과로 본다
- 학습 루프 **다섯 줄**을 보지 않고 쓴다

6.5 숫자를 맞힐 때: MSELoss

다트를 던졌으면 과녁 중심에서 얼마나 빗나갔는지 잰다: 그 **빗나간 정도**를 숫자 하나로 만드는 것이 손실 함수

```
예측 = torch.tensor([2.5, 0.0, 2.0, 8.0])     # 집값 예측(억 원, 예제용)
정답 = torch.tensor([3.0, -0.5, 2.0, 7.0])
손실함수 = nn.MSELoss()
print("MSE :", 손실함수(예측, 정답))
print("손으로:", ((예측 - 정답) ** 2).mean())
print("MAE :", nn.L1Loss()(예측, 정답))

어긋난_예측 = 예측.unsqueeze(1)    # 모델 출력은 보통 (묶음, 1)
print("어긋난 모양:", 손실함수(어긋난_예측, 정답))
print("맞춘 모양 :", 손실함수(어긋난_예측.squeeze(1), 정답))
```

```
MSE : tensor(0.3750)
손으로: tensor(0.3750)
MAE : tensor(0.5000)
어긋난 모양: tensor(16.1562)
맞춘 모양 : tensor(0.3750)
```

(4, 1)과 (4,)가 **(4, 4)**로 펼쳐져 예측 하나를 정답 넷과 비교: 오류 없이 엉뚱한 값

손실에 넣기 전에 **두 모양이 같은지** 확인

6.5 종류를 맞힐 때: CrossEntropyLoss

```
로짓 = torch.tensor([[2.0, 1.0, 0.1],     # 사진 2장 × 클래스 3개
                     [0.5, 0.5, 3.0]])
정답 = torch.tensor([0, 2])               # 클래스 번호(int64)
손실함수 = nn.CrossEntropyLoss()
print("CrossEntropy:", 손실함수(로짓, 정답))

# 손으로: 정답 칸의 확률에 -log를 씌워 평균
확률 = torch.softmax(로짓, dim=1)
정답_확률 = 확률[torch.arange(2), 정답]
print("정답 칸 확률:", 정답_확률)
print("손으로      :", -torch.log(정답_확률).mean())
```

```
CrossEntropy: tensor(0.2845)
정답 칸 확률: tensor([0.6590, 0.8590])
손으로      : tensor(0.2845)
```

정답 확률	손실
0.99	0.010
0.9	0.105
0.5	0.693
0.1	**2.303**
0.01	4.605

넣는 것: **로짓** (묶음, 클래스 수)와 **정수 정답** (묶음,) · 틀린 답을 확신할수록 손실이 치솟는다

6.5 소프트맥스를 미리 씌우면 생기는 일

```
for 점수 in [1.0, 3.0, 10.0]:
    로짓 = torch.tensor([[점수, 0.0, 0.0]])  # 점점 더 확신하는 모델
    바르게 = 손실함수(로짓, 정답)
    잘못 = 손실함수(torch.softmax(로짓, dim=1), 정답)
```

정답 점수	로짓을 넣으면	확률을 넣으면
1.0	0.5514	0.8711
3.0	0.0949	0.6113
10.0	0.0001	**0.5515**

소프트맥스를 **두 번** 씌운 셈: 완벽하게 맞혀도 0.55 밑으로 안 내려가고, 기울기도 흐려진다. **모델은 로짓을 내고, 손실 함수가 알아서**

문제	마지막 층의 출력	손실 함수	정답의 모양과 자료형
회귀(숫자 하나)	nn.Linear(..., 1) 그대로	MSELoss, L1Loss	(묶음, 1) 실수
이진 분류	nn.Linear(..., 1) 로짓	BCEWithLogitsLoss	(묶음, 1) 실수 0. 또는 1.
다중 분류	nn.Linear(..., 클래스 수) 로짓	CrossEntropyLoss	(묶음,) 정수 클래스 번호

6.6 옵티마이저: step()이 하는 일

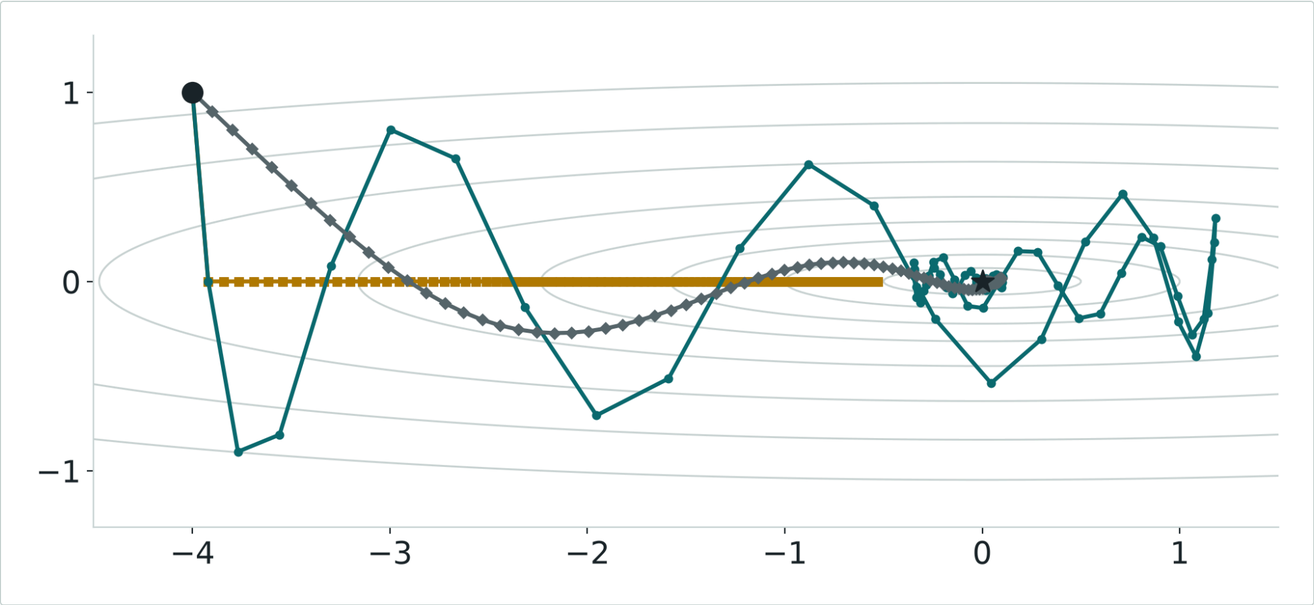
```
층 = nn.Linear(2, 1)
옵티마이저 = torch.optim.SGD(층.parameters(), lr=0.1)
손실 = nn.MSELoss()(층(x), y)
손실.backward()                                  # 기울기 계산(5장)
이전 = 층.weight.detach().clone()
기울기 = 층.weight.grad.clone()
옵티마이저.step()                                # 파라미터 고치기
```

```
고치기 전: tensor([[-0.0053,  0.3793]])
기울기   : tensor([[-1.6573, -3.3145]])
고친 뒤  : tensor([[0.1604, 0.7108]])
이전 - 0.1 × 기울기와 같은가: True
```

5장에서 손으로: no_grad 안에서 **w -= 학습률 * w.grad**, 끝나면 w.grad.zero_()
→ 이제 **step()**과 **zero_grad()** 두 줄

만들 때 **parameters()**를 넘겨 '고칠 것은 이것들'이라고 알려 준다. 리스트에 담겨 빠진 층은 옵티마이저가 존재조차 모른다

6.6 길쭉한 골짜기를 내려가는 세 가지 방법



■ SGD lr=0.01 · ● SGD+momentum lr=0.01 · ◆ Adam lr=0.1 · ★ 바닥 (0, 0)

옵티마이저	10단계	100단계
SGD 0.01	10.68	0.2814
SGD 0.03	5.243e+07	**inf**
모멘텀 0.01	3.046	**0.0003772**
Adam 0.1	9.35	0.0008905

가파른 쪽에 맞춰 학습률을 작게 잡으면 완만한 쪽이 느리다. **모멘텀**은 굴러가는 공, Adam은 파라미터마다 걸음 크기를 따로

7차시 · 6.6 골짜기를 내려가는 세 가지 방법 · 예제 06-nn-module/b15_optim_compare.py와 같은 설정으로 발자취를 그린 그림

6.7 다트 연습 한 번에 다섯 동작

다트 연습	학습 루프 한 줄
1. 지난번 고친 기억 털기	옵티마이저.zero_grad()
2. 다트 던지기	출력 = 모델(x)
3. 빗나간 정도 재기	손실 = 손실함수(출력, y)
4. 어느 쪽으로 고칠지 생각	손실.backward()
5. 자세 고치기	옵티마이저.step()

```
[2930.5, 327.2, 47.7, 17.7, 14.5, 14.1] → 14.1
w = 12.640, b = 52.435
```

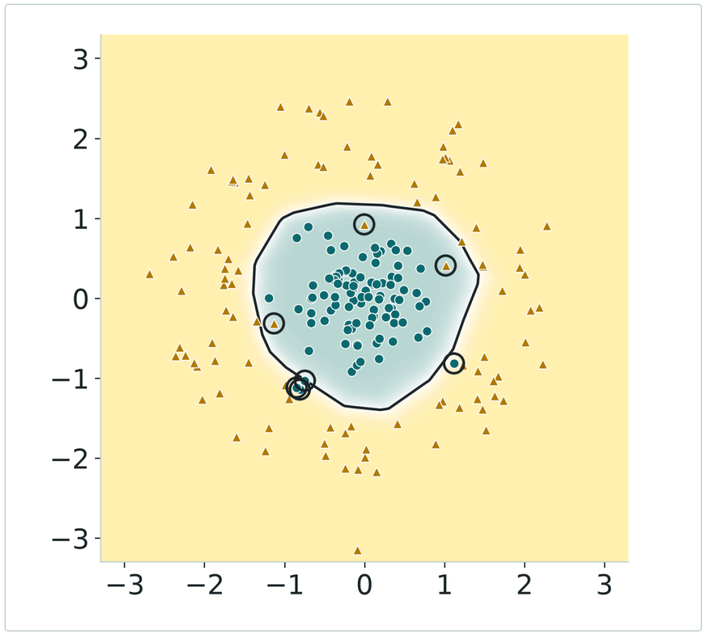
5장 5.7절 아이스크림 회귀를 nn.Linear(1, 1) · MSELoss · SGD로 옮겼더니 손실이 **한 자리도 다르지 않다**

> **zero_grad()**를 빼먹으면 오류 없이 학습만 조용히 망가진다

7차시 · 6.7 표준 학습 루프 · 예제 06-nn-module/b16_from_ch5.py (출력)

6.7 미니배치로 분류 모델 학습하기

```
모델 = nn.Sequential(nn.Linear(2, 16), nn.ReLU(), nn.Linear(16, 16),
                     nn.ReLU(), nn.Linear(16, 2))
손실함수 = nn.CrossEntropyLoss()
옵티마이저 = torch.optim.Adam(모델.parameters(), lr=0.01)
for 에폭 in range(1, 21):
    모델.train()
    섞기 = torch.randperm(800)
    for 시작 in range(0, 800, 32):
        번호 = 섞기[시작:시작 + 32]
        xb, yb = X_학습[번호], y_학습[번호]
        옵티마이저.zero_grad()            # 1) 지난 기울기 지우기
        출력 = 모델(xb)                    # 2) 순전파: 예측하기
        손실 = 손실함수(출력, yb)          # 3) 얼마나 틀렸나 재기
        손실.backward()                    # 4) 역전파: 고칠 방향 계산
        옵티마이저.step()                  # 5) 파라미터 고치기
```




```
에폭  1: 학습 손실 0.5459, 시험 정확도 95.5%
에폭 20: 학습 손실 0.0563, 시험 정확도 96.5%
```

7차시 · 6.7 미니배치로 분류 모델 학습하기 · 예제 06-nn-module/b17_train_loop.py (핵심 부분, 그림은 같은 설정으로 다시 학습)

직접 실행 10분

1. b15_optim_compare.py의 후보에 SGD lr=0.019와 lr=0.021 두 줄을 더해 실행한다. 어디서 갈리나?
2. b16_from_ch5.py에서 **옵티마이저.zero_grad()** 줄을 지우고 실행한다. 손실은 어떻게 움직이나?
3. b17_train_loop.py에서 **nn.ReLU()를 모두 빼고** 실행한다. 시험 정확도는?

```
cd examples/vol2/06-nn-module && uv run python b15_optim_compare.py
```

클래스 10개 분류 모델, 학습 전 첫 손실이 무엇에 가까우면 정상일까요?

Q

가 0

나 0.693

다 2.303

라 10

7차시 정리

회귀는 **MSELoss**(모양 맞추기), 다중 분류는 **CrossEntropyLoss**에 로짓과 정수 정답, 이진 분류는 BCEWithLogitsLoss

옵티마이저 **step()**은 기울기 방향으로 고치기. SGD·모멘텀·Adam, 가장 큰 손잡이는 학습률

zero_grad → 모델(x) → 손실 → backward → step, 에폭과 미니배치로 반복

다음 차시: 평가 루프, 장치, 저장, 그리고 옷 사진 7만 장 분류기

8차시

08

평가, 장치, 저장, 그리고 옷 사진 분류기

책 6.8 ~ 6.11절

학습 목표

- **eval()**과 **no_grad()**가 각각 무엇을 끄는지 구분한다
- 모델을 장치로 옮기고 state_dict로 저장하고 불러온다
- FashionMNIST 분류기를 끝까지 만들고 **틀린 것**을 들여다본다

6.8 학습 모드와 평가 모드

```
모델 = nn.Sequential(nn.Linear(4, 4), nn.Dropout(p=0.5))
x = torch.ones(1, 4)

모델.train()                    # 학습 모드(처음 만들면 이 모드)
print(모델(x))
print(모델(x))                   # 같은 입력인데 결과가 다르다

모델.eval()                     # 평가 모드
print(모델(x))
print(모델(x))
```

```
학습 모드: True
[[-0.6347,  1.2561, -1.0009, -0.0000]]
[[-0.6347,  1.2561, -1.0009, -2.4283]]
평가 모드: False False
[[-0.3173,  0.6281, -0.5004, -1.2141]]
[[-0.3173,  0.6281, -0.5004, -1.2141]]
```

드롭아웃: 학습 중에 일부 출력을 무작위로 끈다. 살아남은 값은 두 배(-0.6347 = 2 × -0.3173)

학습 전에는 **모델.train()**, 평가 전에는 **모델.eval()**: 안쪽 층까지 모드가 전달된다

8차시 · 6.8 model.train()과 model.eval() · 예제 06-nn-module/b18_train_eval_mode.py (출력은 tensor(...)와 grad_fn 표시를 줄였다)

6.8 평가 루프의 표준형: eval + no_grad

```
def 정확도_재기(모델: nn.Module, X: torch.Tensor, y: torch.Tensor) -> float:
    모델.eval()                              # 1) 평가 모드로
    with torch.no_grad():                    # 2) 계산 기록 끄기
        예측 = 모델(X).argmax(dim=1)      # 3) 가장 높은 점수의 클래스
    return (예측 == y).float().mean().item()
```

```
기록함    : True AddmmBackward0
기록 안 함: False None
학습 전 정확도: 30%
다시 학습 모드로 되돌렸나: False
```

	하는 일	빼먹으면
모델.eval()	드롭아웃 같은 층의 **동작**을 바꾼다	시험 점수가 매번 달라진다
torch.no_grad()	**기울기 기록**을 끈다	숫자는 같지만 메모리를 더 쓰고 느리다

평가 함수가 끝나도 모델은 **평가 모드에 남는다**: 에폭마다 **모델.train()**을 다시 부른다

8차시 · 6.8 torch.no_grad()와 평가 함수 · 예제 06-nn-module/b19_no_grad.py (핵심 부분)

6.9 모델을 장치로 옮기기

```
장치 = 장치_고르기()                        # cuda → mps → cpu 순서로
모델 = nn.Linear(3, 1)
옮긴_모델 = 모델.to(장치)                    # 모델은 제자리에서 옮겨진다
print("같은 객체인가:", 옮긴_모델 is 모델)
print("파라미터 위치:", next(모델.parameters()).device)
x = torch.rand(2, 3)                        # 데이터는 아직 CPU에
try:
    모델(x)
except RuntimeError as e:
    print("오류:", e)
x = x.to(장치)      # 텐서는 새 텐서를 돌려주니 다시 받는다
print(모델(x).detach().cpu())
```

```
같은 객체인가: True
파라미터 위치: mps:0
오류: Tensor for argument input
  is on cpu but expected on mps
tensor([[-0.4627],
        [-0.5881]])
```

루프 안에서 묶음마다 **x, y = x.to(장치), y.to(장치)**

순서: 모델 만들기 → **장치로 옮기기** → 옵티마이저 만들기

8차시 · 6.9 장치로 옮기기 · 예제 06-nn-module/b20_device.py (핵심 부분, 오류 메시지는 두 줄로 접었다)

6.10 저장과 불러오기: state_dict와 체크포인트

예측용: 모델의 숫자만

```
torch.save(모델.state_dict(), "작은신경망.pt")
새_모델 = 작은신경망(4, 8, 3)        # 같은 설계도 먼저
새_모델.load_state_dict(
    torch.load("작은신경망.pt", weights_only=True))
```

```
층1.weight  (8, 4)    층1.bias  (8,)
층2.weight  (3, 8)    층2.bias  (3,)
불러오기 전 같은가: False
불러온 뒤 같은가  : True
```

은닉 16으로 만든 모델에 불러오면: **size mismatch for 층1.weight** … [8, 4] … [16, 4]

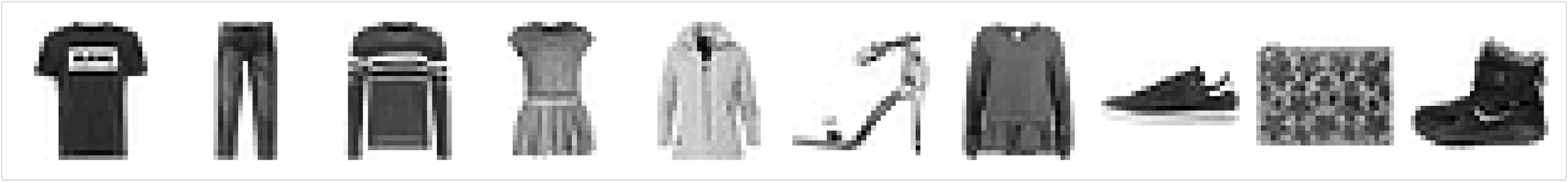
이어서 학습용: 체크포인트

```
torch.save({
    "에폭": 5,
    "모델": 모델.state_dict(),
    "옵티마이저": 옵티마이저.state_dict(),
}, "체크포인트.pt")
상자 = torch.load("체크포인트.pt", weights_only=True)
새_모델.load_state_dict(상자["모델"])
새_옵티마이저.load_state_dict(상자["옵티마이저"])
```

```
이어서 시작할 에폭: 6
옵티마이저가 기억하는 것:
  ['step', 'exp_avg', 'exp_avg_sq']
```

8차시 · 6.10 모델 저장과 불러오기 · 예제 06-nn-module/b21_state_dict.py, b22_checkpoint.py (핵심 부분, 출력 일부를 한 줄로 모았다)

6.11 실습: FashionMNIST 둘러보기



0 T셔츠 1 바지 2 풀오버 3 드레스 4 코트 5 샌들 6 셔츠 7 운동화 8 가방 9 앵클부츠

```
변환 = transforms.ToTensor()      # 사진 → (1, 28, 28) 실수 텐서, 값은 0~1
학습셋 = datasets.FashionMNIST("data", train=True, download=True, transform=변환)
시험셋 = datasets.FashionMNIST("data", train=False, download=True, transform=변환)
```

```
학습 60000 장 / 시험 10000 장
클래스별 학습 사진 수: [6000, 6000, ...]
torch.Size([1, 28, 28]) torch.float32
값 범위 0.0~1.0 정답: 9 Ankle boot
```

잘란도(Zalando)가 MIT 라이선스로 공개한 옷 사진 **7만 장**, 열 클래스가 6,000장씩 고르다: 정확도를 그대로 성적으로 봐도 된다

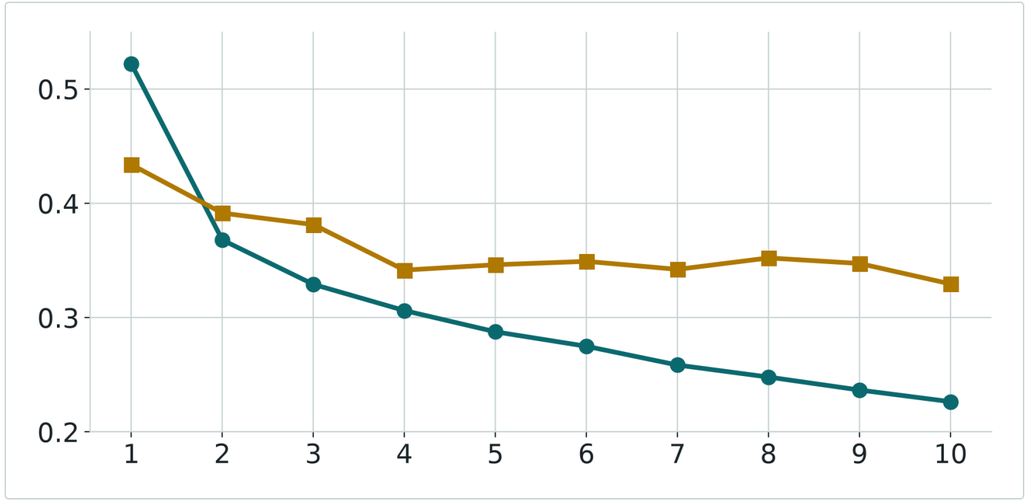
8차시 · 6.11 데이터 둘러보기 · 예제 06-nn-module/b23_fashion_look.py (핵심 부분, 그림은 클래스마다 첫 학습 사진)

6.11 학습시키기: 부품을 모두 조립한다

```
class 의류분류기(nn.Module):
    def __init__(self) -> None:
        super().__init__()
        self.펴기 = nn.Flatten()
        self.층들 = nn.Sequential(
            nn.Linear(28 * 28, 256), nn.ReLU(),
            nn.Linear(256, 128), nn.ReLU(),
            nn.Linear(128, 10),
        )

    def forward(self, x: torch.Tensor) -> torch.Tensor:
        return self.층들(self.펴기(x))

학습_로더 = DataLoader(학습셋, batch_size=64, shuffle=True)
모델 = 의류분류기().to(장치)
옵티마이저 = torch.optim.Adam(모델.parameters(), lr=1e-3)
for 에폭 in range(1, 11):
    학습_손실 = 한_에폭_학습(모델, 학습_로더, 손실함수, 옵티마이저, 장치)
    시험_손실, 정확도 = 평가하기(모델, 시험_로더, 손실함수, 장치)
```

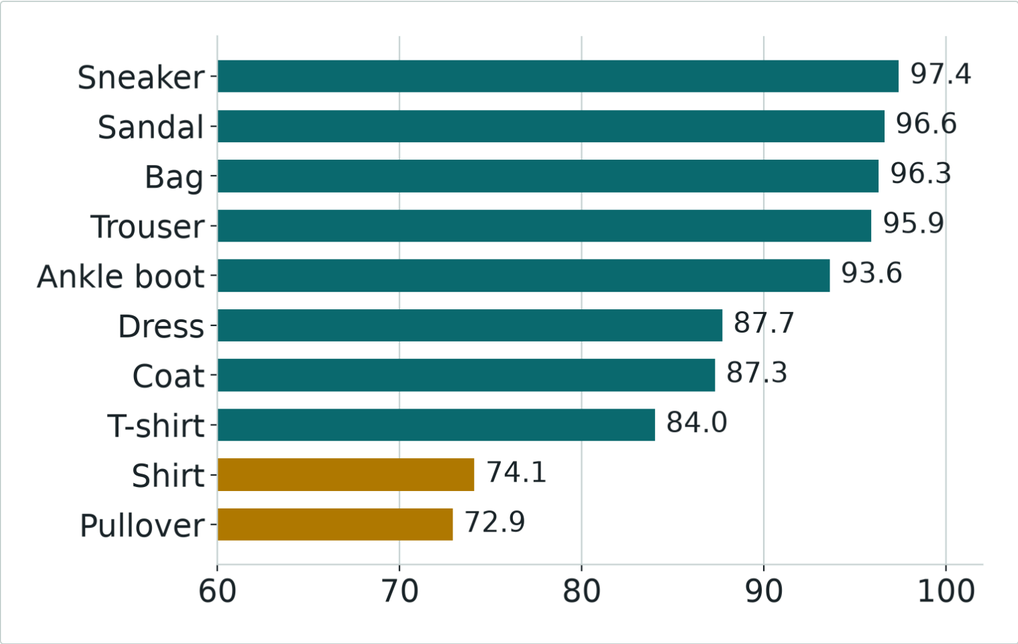


● 학습 손실 · ■ 시험 손실 · 가로는 에폭

```
파라미터 235,146개
에폭  1: ... 시험 정확도 84.04%
에폭 10: ... 시험 정확도 88.58%
```

8차시 · 6.11 학습시키기 · 예제 06-nn-module/b24_fashion_train.py (핵심 부분, 19초) · 그림은 같은 설정으로 다시 학습

6.11 무엇을 틀렸을까



클래스별 정확도(%) · 전체 88.58%

헷갈린 짝	횟수
풀오버 → 코트	172
T셔츠 → 셔츠	116
셔츠 → T셔츠	85



12번 운동화
→ 샌들 66.2%

17번 코트
→ 풀오버 72.8%

23번 앵클부츠
→ 샌들 **99.6%**

8차시 · 6.11 무엇을 틀렸을까 · 예제 06-nn-module/b25_fashion_inspect.py (출력을 그림과 표로)

6.11 옵티마이저 바꿔 보기: 진짜 데이터에서는

같은 모델, 같은 씨앗, 2에폭	시험 정확도
SGD lr=0.01	76.44%
SGD+momentum lr=0.01	84.12%
Adam lr=0.001	**86.07%**
Adam lr=0.1	**33.35%**

골짜기에서는 모멘텀이 1등, 여기서는 Adam: 항상 이기는 옵티마이저는 없다

같은 Adam도 학습률 0.1이면 무너진다: **종류보다 학습률**

앞의 2에폭째 86.26%와 다른 이유: 섞는 순서. 0.1~0.2%p 차이로 우열을 가리지 않는다

직접 실행 10분

1. b23_fashion_look.py를 실행해 데이터를 받고 첫 사진의 글자 그림을 본다 (처음엔 약 30MB, 20초 안팎)
2. b24_fashion_train.py를 복사해 은닉층을 **256·128 → 64·32**로, 저장 파일 이름을 **fashion_small.pt**로 바꾼다. 파라미터 수와 10에폭 정확도를 예상하고 실행한다
3. 2번 파일의 학습 루프 바로 앞에서 **평가하기**를 한 번 불러, 학습 전 시험 손실이 2.303 근처인지 확인한다

```
cd examples/vol2/06-nn-module && cp b24_fashion_train.py 작은_학습.py && uv run python 작은_학습.py
```

8차시 · 직접 실행 · 2·3번은 한 번 실행으로 함께 확인한다 (약 20초)

학습한 모델로 시험 점수를 잴 때 해야 할 것은?

가 모델.eval()만

나 torch.no_grad()만

다 모델.eval()과 torch.no_grad() 둘 다

라 모델.train()과 torch.no_grad()

8차시 정리

평가는 **eval() + no_grad()**, 다시 학습할 때 train(). 모델은 .to(장치)로 제자리에서, 데이터는 묶음마다 옮긴다

숫자는 **state_dict**, 설계도는 코드. 이어서 학습하려면 옵티마이저 상태까지 체크포인트로

사진을 펴서 보는 MLP로 **88.58%**: 풀오버·셔츠가 약하고, 시험 손실은 4에폭부터 멈췄다

다음 차시: DataLoader 안을 열어 본다 · Dataset, TensorDataset, DataLoader

9차시

09

Dataset과 DataLoader

책 7장 들어가며 ~ 7.3절

학습 목표

- 데이터를 한꺼번에 메모리에 올리지 않는 이유를 숫자로 말한다
- __len__과 __getitem__으로 Dataset을 만든다
- DataLoader의 batch_size·shuffle·drop_last와 배치를 합치는 규칙을 결과로 본다

7장 주방은 재료를 한꺼번에 꺼내지 않는다

```
사진 한 장: 숫자 150,528개, 0.57MB
사진     1,000장:       0.6GB
사진   100,000장:      56.1GB
사진 1,000,000장:     560.8GB
```

224 × 224 컬러 사진 한 장은 0.57MB,
10만 장이면 56GB: 노트북 메모리로는
한꺼번에 올릴 수 없다

Dataset · 창고 목록

몇 개인지 알고, n번을 달라면 **그 하나만** 꺼낸다

transform · 손질

꺼낸 샘플을 텐서로 바꾸고, 필요하면 살짝 변형(증강)한다

DataLoader · 쟁반

여러 개를 꺼내 **배치 하나**로 묶어 낸다. 섞기, 일꾼 나누기도

7.1 Dataset: 번호를 주면 샘플 하나를 주는 약속

```
class 제곱수_데이터셋(Dataset):
    def __init__(self, n: int) -> None:
        self.n = n                          # 재료 목록만 기억해 둔다

    def __len__(self) -> int:
        return self.n                       # 샘플이 모두 몇 개인가

    def __getitem__(self, i: int) -> tuple[int, int]:
        if i >= self.n:
            raise IndexError(f"{i}번 샘플은 없습니다")
        print(f"  ({i}번 샘플을 지금 만듭니다)")
        return i, i * i                     # 그때 만들어 돌려준다

데이터 = 제곱수_데이터셋(4)
print("샘플 수:", len(데이터))
print("3번 샘플:", 데이터[3])
for 입력, 정답 in 데이터:
    print("반복:", 입력, 정답)
```

```
샘플 수: 4
  (3번 샘플을 지금 만듭니다)
3번 샘플: (3, 9)
  (0번 샘플을 지금 만듭니다)
반복: 0 0
  (1번 샘플을 지금 만듭니다)
반복: 1 1
  ...
반복: 3 9
```

__init__은 가볍게, 무거운 일은 __getitem__에서

7.2 TensorDataset: 이미 텐서로 있을 때

```
torch.manual_seed(0)
X = torch.randn(6, 3)                       # 샘플 6개 × 특성 3개
y = torch.tensor([0, 1, 0, 1, 1, 0])      # 샘플마다 정답 하나

데이터 = TensorDataset(X, y)
print("샘플 수:", len(데이터))
x0, y0 = 데이터[0]        # 0번 샘플: (특성 3개, 정답)
print(x0, y0)
특성들, 정답들 = 데이터[1:3]                # 슬라이스로 여러 개도
print(특성들.shape, 정답들)

try:
    TensorDataset(X, torch.tensor([0, 1]))   # 샘플 수가 다르면?
except AssertionError as e:
    print("오류:", e)
```

```
샘플 수: 6
tensor([-1.1258, -1.1524,  0.5667]) tensor(0)
torch.Size([2, 3]) tensor([1, 0])
오류: Size mismatch between tensors
```

같은 번호끼리 짝지어 준다: X의 0번 줄 + y의 0번 값 = 0번 샘플

개수가 어긋나면 **바로 오류**: 실수를 일찍 잡는다

7.3 DataLoader: 배치로 묶어 나르기

```
X = torch.arange(10).float().unsqueeze(1) * 10  # 0, 10, ..., 90
y = torch.arange(10)                    # 정답 = 샘플 번호
데이터 = TensorDataset(X, y)

로더 = DataLoader(데이터, batch_size=4)
print("배치 수:", len(로더))
for xb, yb in 로더:
    print(tuple(xb.shape), yb.tolist())

버리는_로더 = DataLoader(데이터, batch_size=4, drop_last=True)

g = torch.Generator().manual_seed(0)   # 섞는 순서도 씨앗으로
섞는_로더 = DataLoader(데이터, batch_size=4, shuffle=True, generator=g)
for 에폭 in range(2):
    print(f"에폭 {에폭}:", [yb.tolist() for _, yb in 섞는_로더])
```

```
배치 수: 3
(4, 1) [0, 1, 2, 3]
(4, 1) [4, 5, 6, 7]
(2, 1) [8, 9]
drop_last=True 배치 수: 2
에폭 0: [[3, 7, 5, 2], [0, 8, 1, 6], [9, 4]]
에폭 1: [[1, 3, 2, 7], [5, 9, 0, 6], [4, 8]]
```

- **batch_size**: 남은 2개도 작은 배치로
- **drop_last**: 모자란 마지막 배치 버리기
- **shuffle**: 에폭마다 새로 섞기
- **generator**: 섞는 순서 재현

7.3 DataLoader의 옵션, 어디서 만나나

옵션	하는 일	나오는 곳
batch_size	배치 하나에 넣을 샘플 수	7.3 (오늘)
shuffle	에폭마다 순서 섞기 (학습용 True, 검증용 False)	7.3 (오늘)
drop_last	모자란 마지막 배치 버리기	7.3 (오늘)
generator	섞기·뽑기에 쓸 난수 씨앗	7.3, 7.8
collate_fn	샘플 목록을 배치 하나로 합치는 함수	7.7 (3일차)
sampler, batch_sampler	어떤 순서로 몇 번 샘플을 뽑을지 직접 정하기	7.7, 7.9 (3일차)
num_workers	샘플을 꺼내는 일꾼 프로세스 수	7.10 (3일차)
persistent_workers	에폭이 끝나도 일꾼을 살려 두기	7.10 (3일차)
pin_memory	CPU→GPU 복사를 빠르게 하는 고정 메모리 (CUDA)	7.10 (3일차)

7.3 샘플을 배치로 합치는 규칙

```
def __getitem__(self, i: int) -> dict:
    return {"이름": self.이름[i], "점수": self.점수[i], "합격": self.합격[i]}

로더 = DataLoader(학생_데이터셋(), batch_size=3)
배치 = next(iter(로더))                        # 첫 배치만 꺼내 보기
```

```
샘플 하나: {'이름': '가온', '점수': tensor([80., 70.]), '합격': 1}
이름: list ['가온', '나래', '다온']
점수: Tensor tensor([[80., 70.],
        [92., 88.],
        [75., 95.]])
합격: Tensor tensor([1, 1, 1])
```

샘플 속 값	배치에서는
텐서	새 차원으로 쌓기 (stack)
숫자	텐서로 모으기
문자열	리스트로 모으기

생김새는 그대로, 안쪽 값만 묶는다: 튜플이면 튜플, 딕셔너리면 딕셔너리. 이 규칙이 **collate**(모아 맞추기)

7.3 번호가 없는 데이터: IterableDataset

```
class 주문_흐름(IterableDataset):
    def __init__(self, 주문_수: int) -> None:
        self.주문_수 = 주문_수

    def __iter__(self) -> Iterator[torch.Tensor]:
        for 번호 in range(self.주문_수):   # 실제로는 파일을 한 줄씩 읽는 자리
            yield torch.tensor([번호, 번호 * 1000])

로더 = DataLoader(주문_흐름(7), batch_size=3)
for 배치 in 로더:
    print(배치.tolist())
DataLoader(주문_흐름(7), batch_size=3, shuffle=True)   # 오류
```

```
[[0, 0], [1, 1000], [2, 2000]]
[[3, 3000], [4, 4000], [5, 5000]]
[[6, 6000]]
오류: DataLoader with IterableDataset:
  expected unspecified shuffle option
```

	맵 스타일 Dataset	**IterableDataset**
만드는 것	__len__, __getitem__	__iter__ (yield)
shuffle	된다	안 된다 (직접 모아 섞기)
언제	**기본**	끝없이 쌓이는 기록, 거대한 로그

9차시 · 7.3 IterableDataset · 예제 07-data-pipeline/b06_iterable.py (핵심 부분, 오류는 try로 잡아 앞부분만 찍는다)

직접 실행 10분

1. b04_dataloader.py의 batch_size=4 두 군데를 3으로 바꾸기 전에, 배치 수와 drop_last=True 배치 수를 예상하고 실행한다
2. b03_tensordataset.py에서 가중치 텐서를 하나 더 넣어 **TensorDataset(X, y, 가중치)**로 바꾸고 그대로 실행한다. 어느 줄이 왜 깨지나?
3. b05_default_collate.py의 샘플에 **"평균": float(self.점수[i].mean())**을 더하면 배치에서 어떤 자료형이 될까?

```
cd examples/vol2/07-data-pipeline && uv run python b04_dataloader.py
```

DataLoader에 넘길 맵 스타일 Dataset이 꼭 가져야 하는 두 메서드는?

Q

가 __init__과 forward

나 __len__과 __getitem__

다 __iter__와 __next__

라 fit과 transform

9차시 정리

Dataset은 창고 목록: __len__과 __getitem__, 무거운 일은 꺼낼 때. 텐서가 이미 있으면 TensorDataset

DataLoader는 쟁반: batch_size, shuffle(학습용), drop_last, generator로 재현

배치는 샘플의 **생김새를 유지**하며 묶인다: 텐서는 쌓고, 숫자는 텐서로, 문자열은 리스트로

다음 차시: 내 CSV 파일과 그림 폴더로 Dataset 만들기, transforms와 증강

10차시

10

CSV·그림 폴더 Dataset과 transforms

책 7.4 ~ 7.6절

학습 목표

- CSV 파일과 그림 폴더를 읽는 Dataset을 직접 만든다
- 표준화·정규화 기준은 **학습 데이터에서만** 구한다
- transforms v2로 손질하고, 증강이 숫자를 어떻게 바꾸는지 본다

7.4 CSV 파일을 읽는 Dataset

```
['무게,당도,껍질,등급', '114.3,10.5,2.09,상', '82.9,10.2,2.77,중', '92.4,12.6,2.56,상']
```

```
class 귤_데이터셋(Dataset):
    특성_열 = ["무게", "당도", "껍질"]
    등급_번호 = {"하": 0, "중": 1, "상": 2}

    def __init__(self, 경로: Path, 평균: torch.Tensor | None = None,
                 표준편차: torch.Tensor | None = None) -> None:
        표 = pd.read_csv(경로)
        self.X = torch.tensor(표[self.특성_열].to_numpy(), dtype=torch.float32)
        self.y = torch.tensor(표["등급"].map(self.등급_번호).to_numpy())
        self.평균 = self.X.mean(dim=0) if 평균 is None else 평균
        self.표준편차 = self.X.std(dim=0) if 표준편차 is None else 표준편차

    def __getitem__(self, i: int) -> tuple[torch.Tensor, torch.Tensor]:
        x = (self.X[i] - self.평균) / self.표준편차   # 꺼낼 때 표준화
        return x, self.y[i]
```

10차시 · 7.4 CSV 파일 · 예제 07-data-pipeline/b07_make_csv.py (출력), b08_csv_dataset.py (핵심 부분)

7.4 검증 데이터는 학습 기준으로 표준화한다

```
학습 = 귤_데이터셋(폴더 / "학습.csv")
검증 = 귤_데이터셋(폴더 / "검증.csv", 학습.평균, 학습.표준편차)  # 학습 기준을 그대로 넘긴다
xb, yb = next(iter(DataLoader(학습, batch_size=8)))
```

```
샘플 수: 160 40
학습 기준 평균: tensor([86.9087, 10.7006,  2.5896])
0번 샘플: tensor([ 2.0863, -0.1422, -1.1424]) tensor(2)
배치 모양: (8, 3) (8,) torch.int64
등급별 개수: [46, 62, 52]
```

검증의 평균을 쓰면 시험 문제를 **미리 엿본** 셈: 2장의 **데이터 누수**

scikit-learn에서 fit은 학습에만, transform은 양쪽에 하던 것과 같은 원칙

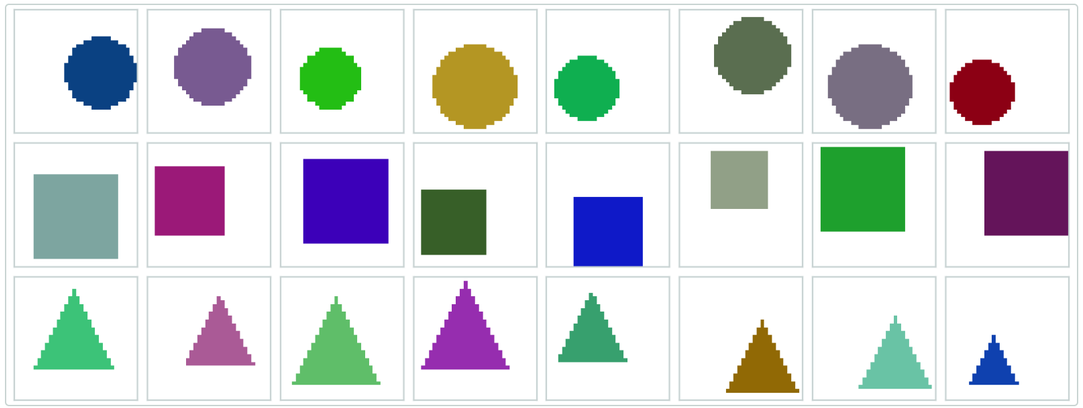
새 데이터셋을 만들면 **클래스별 개수**부터 센다: torch.bincount → 하·중·상 46, 62, 52

7.5 그림 폴더: 폴더 이름이 곧 정답

```
data/shapes/
 ├── circle/     000.png, 001.png, ...
 ├── square/     000.png, 001.png, ...
 └── triangle/   000.png, 001.png, ...
```

```
circle: 20개 ['000.png', '001.png', '002.png'] ...
square: 20개 ['000.png', '001.png', '002.png'] ...
triangle: 20개 ['000.png', '001.png', '002.png'] ...
```

32 × 32 흰 바탕에 동그라미·네모·세모를 무작위 색·크기·위치로. **도형_만들기.py**로 저장해 3일차 실습에서도 쓴다



위부터 circle · square · triangle, 각 000~007번

7.5 그림 폴더를 읽는 Dataset

```
class 그림폴더_데이터셋(Dataset):
    def __init__(self, 뿌리: Path) -> None:
        self.클래스들 = sorted(p.name for p in 뿌리.iterdir() if p.is_dir())
        self.클래스_번호 = {이름: i for i, 이름 in enumerate(self.클래스들)}
        self.목록 = [(경로, self.클래스_번호[이름])      # 그림은 읽지 않고 목록만
                     for 이름 in self.클래스들
                     for 경로 in sorted((뿌리 / 이름).glob("*.png"))]
    def __getitem__(self, i: int) -> tuple[torch.Tensor, int]:
        경로, 정답 = self.목록[i]
        with Image.open(경로) as 그림:                    # 필요한 그 순간에 연다
            배열 = np.array(그림.convert("RGB"))           # (세로, 가로, 색), 0~255
        텐서 = torch.from_numpy(배열).permute(2, 0, 1)   # (색, 세로, 가로)
        return 텐서.float() / 255, 정답
```

```
클래스: {'circle': 0, 'square': 1, 'triangle': 2} 샘플 수: 60
25번: (3, 32, 32) torch.float32 정답 1 경로 005.png
ImageFolder: {'circle': 0, 'square': 1, 'triangle': 2} 60
25번: Image (32, 32) 정답 1 data/shapes/square/005.png
```

정렬해야 컴퓨터마다 번호가 같다. ImageFolder는 같은 일을 한 줄로, 단 그림은 **Pillow 그대로**

7.6 transforms: 꺼낼 때 손질하기

```
배열 = np.array([[[255, 0, 0], [0, 255, 0], [0, 0, 255]],
                 [[255, 255, 255], [128, 128, 128], [0, 0, 0]]],
                dtype=np.uint8)
그림 = Image.fromarray(배열)

텐서 = v2.ToImage()(그림)     # PIL → (색, 세로, 가로)
실수 = v2.ToDtype(torch.float32, scale=True)(텐서)   # 0~255 → 0~1
정규화 = v2.Normalize(mean=[0.5, 0.5, 0.5], std=[0.5, 0.5, 0.5])(실수)

한번에 = v2.Compose([v2.ToImage(),
                     v2.ToDtype(torch.float32, scale=True),
                     v2.Normalize(mean=[0.5] * 3, std=[0.5] * 3)])
```

```
ToImage : (3, 2, 3) torch.uint8
tensor([[255,   0,   0],
        [255, 128,   0]], dtype=torch.uint8)
ToDtype : torch.float32
tensor([[1.0000, 0.0000, 0.0000],
        [1.0000, 0.5020, 0.0000]])
Normalize:
tensor([[ 1.0000, -1.0000, -1.0000],
        [ 1.0000,  0.0039, -1.0000]])
Compose와 같은가: True
```

scale=True를 빼먹으면 자료형만 바뀌고 값은 0~255 그대로

Normalize: (값 − 평균) / 표준편차, 기준은 **학습 데이터에서**

10차시 · 7.6 변환 단계를 숫자로 따라가기 · 예제 07-data-pipeline/b11_transform_steps.py (핵심 부분, 빨강 채널만 출력)

7.6 자주 쓰는 변환

변환	하는 일	증강인가
v2.ToImage()	Pillow 그림·NumPy 배열 → (색, 세로, 가로) 텐서	아니요
v2.ToDtype(torch.float32, scale=True)	실수로 바꾸고 0~1로 맞추기	아니요
v2.Normalize(mean, std)	채널마다 (값 – 평균) / 표준편차	아니요
v2.Resize(크기)	크기 바꾸기	아니요
v2.CenterCrop(크기)	가운데 잘라 내기	아니요
v2.RandomHorizontalFlip(p)	확률 p로 좌우 뒤집기	**예**
v2.RandomCrop(크기, padding)	테두리를 두르고 무작위 위치 잘라 내기	**예**
v2.RandomResizedCrop(크기)	무작위 크기·위치로 잘라 원하는 크기로	**예**
v2.RandomRotation(각도)	무작위 각도로 돌리기	**예**
v2.ColorJitter(…)	밝기·대비·채도를 무작위로 바꾸기	**예**

7.6 데이터 증강: 같은 그림을 조금씩 다르게

원본

```
[[ 1,  2,  3,  4],
 [ 5,  6,  7,  8],
 [ 9, 10, 11, 12]]
```

RandomHorizontalFlip(p=1.0)

```
[[ 4,  3,  2,  1],
 [ 8,  7,  6,  5],
 [12, 11, 10,  9]]
```

RandomVerticalFlip(p=1.0)

```
[[ 9, 10, 11, 12],
 [ 5,  6,  7,  8],
 [ 1,  2,  3,  4]]
```

RandomCrop((3, 4), padding=1) 1

```
[[ 0,  5,  6,  7],
 [ 0,  9, 10, 11],
 [ 0,  0,  0,  0]]
```

RandomCrop((3, 4), padding=1) 2

```
[[ 0,  1,  2,  3],
 [ 0,  5,  6,  7],
 [ 0,  9, 10, 11]]
```

RandomHorizontalFlip(p=0.5)

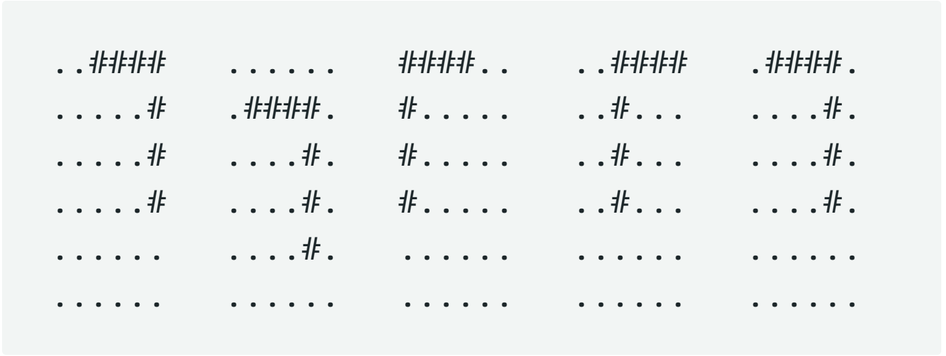
1000번 중 **493번** 뒤집혔다

고양이는 뒤집어도, 조금 잘라도 고양이: **정답이 바뀌지 않는 범위**에서 살짝 바꿔 보여 준다

10차시 · 7.6 데이터 증강 · 예제 07-data-pipeline/b12_augment_numbers.py (출력, tensor와 dtype 표시를 줄였다)

7.6 증강은 데이터 개수를 늘리지 않는다

```
증강 = v2.Compose([v2.RandomHorizontalFlip(p=0.5),
                   v2.RandomCrop(size=6, padding=1)])
데이터 = 글자_데이터셋(transform=증강)     # 그림은 ㄱ 한 장
그려_보기([데이터[0] for _ in range(5)])   # 같은 0번을 다섯 번
```



길이는 그대로 1, 한 에폭에 한 번. 대신 **에폭마다 다른 모습**: 100 에폭이면 100가지 변형

좌우가 뒤집힌 ㄱ은 ㄱ이 아니다. 6을 상하로 뒤집으면 9. 증강은 **학습 데이터에만**

직접 실행 10분

1. b08_csv_dataset.py에서 검증셋을 **자기 평균으로** 만든 것(귤_데이터셋(폴더 / "검증.csv"))과 비교한다. 검증 0번 샘플이 얼마나 달라지나?
2. b11_transform_steps.py의 첫 ToDtype에서 **scale=True**를 지우고 실행한다. Normalize 결과는?
3. b13_augment_dataset.py에서 **RandomHorizontalFlip**을 빼고 실행한다. 다섯 장은 어떻게 달라지나?

```
cd examples/vol2/07-data-pipeline && uv run python b08_csv_dataset.py
```

손글씨 숫자(0~9) 분류 모델에 쓰면 안 되는 증강은?

Q

가 RandomCrop(28, padding=2)

나 RandomRotation(10)

다 RandomVerticalFlip(p=0.5)

라 ColorJitter(밝기를 조금)

10차시 정리

CSV는 __init__에서 읽어 두고 꺼낼 때 표준화, 그림 폴더는 **경로 목록만** 들고 꺼낼 때 연다

표준화·정규화 기준은 **학습 데이터에서만** 구해 검증에 넘긴다

transforms v2: ToImage → ToDtype(scale=True) → Normalize. 증강은 **정답이 그대로인 범위**에서, 학습에만

다음 시간(3일차): collate_fn, 학습/검증 나누기, 불균형, num_workers, 도형 파이프라인 실습

2일차에 배운 것

1. 층은 **입력 @ 가중치 + 편향**, 사이의 ReLU가 직선을 구부린다
2. 모델은 로짓을 내고 **손실 함수**가 알아서, 고치는 건 옵티마이저
3. **다섯 줄** 학습 루프와 eval + no_grad 평가 루프, state_dict 저장
4. Dataset은 창고 목록, DataLoader는 쟁반
5. 꺼낼 때 손질하고, 증강은 정답이 그대로인 범위에서

내일 3일차

- 7장 나머지와 도형 파이프라인 실습
- 과적합 진단, weight decay, 드롭아웃
- 배치 정규화, 초기화, 스케줄러
- 조기 종료, 클리핑, 재현성, 디버깅

2일차 마무리 · 복습: 책 6장 전체, 7장 들어가며 ~ 7.6절